

Data Preprocessing with Python

Cleaning, Transforming, and Standardizing Data for Machine Learning

DSAI1005 - Data Analysis with Python

Department of Data Science and Artificial Intelligence
National Economics University

2026

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Motivation

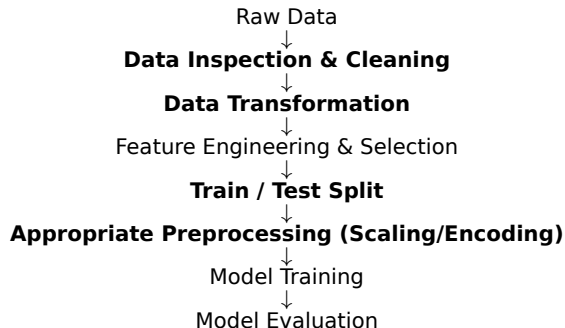
In practice, raw data is rarely ready to be fed directly into machine learning models:

- **Missing values:** caused by user omissions or data transmission errors.
- **Outliers:** could be clerical errors or rare, high-value observations.
- **Improper data types:** numeric variables stored as strings, incorrect datetime formats.
- **Categorical variables:** text labels that need to be transformed into numerical formats.
- **Scale discrepancies:** income in millions vs. age under 100.
- **Class imbalance:** fraud detection or medical diagnoses with low positive rates.

Core Principle

Garbage In, Garbage Out: Complex machine learning algorithms trained on dirty data produce unreliable and biased results.

Position in the Machine Learning Workflow



Key Benefits:

- Improves the quality of model inputs.
- Mitigates the harmful impact of noise.
- Ensures distance-based and gradient-based algorithms work correctly.
- **Prevents Data Leakage:** Eliminates leakage of test information into the training phase.
- Makes model decisions more interpretable.

Why Do Algorithms Need Preprocessing?

Many machine learning algorithms are sensitive to feature scales:

- Consider two features: Income (5,000,000 to 100,000,000) and Age (18 to 70).
- Using Euclidean distance:

$$d = \sqrt{(\text{Income}_1 - \text{Income}_2)^2 + (\text{Age}_1 - \text{Age}_2)^2}$$

- Income completely **dominates** the distance calculation, rendering Age irrelevant!

Heavily Impacted Algorithms:

- k-Nearest Neighbors (k-NN), k-Means Clustering
- Support Vector Machines (SVM)
- Regularized Regression (Ridge, Lasso, ElasticNet)
- Artificial Neural Networks (Deep Learning)

Learning Objectives

By the end of this lecture, students will be able to:

- 1 Inspect data structures, types, and missing values using Pandas.
- 2 Differentiate between **missing values**, **outliers**, and invalid inputs.
- 3 Utilize descriptive statistics and visualizations (boxplots, histograms) to evaluate data quality.
- 4 Apply the IQR method to detect outliers and evaluate appropriate treatment strategies.
- 5 Encode categorical variables effectively using **One-Hot Encoding**.
- 6 Differentiate and correctly apply **Normalization** (Min-Max) and **Standardization** (Z-score).
- 7 Understand the **data leakage prevention principle**: Preprocessing transformations must be learned exclusively from the **Training set**.
- 8 Construct end-to-end preprocessing workflows using Scikit-Learn's **ColumnTransformer** and **Pipeline**.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure**
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Step 1: Importing Libraries and Loading Data

Demonstration dataset: `diabetes.csv` (Pima Indians Diabetes Dataset).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.impute import SimpleImputer

# Load dataset
df = pd.read_csv("diabetes.csv")
df.head()
```

`df.head()` displays the first 5 records: quick inspection of column names, data representations, value types, and obvious anomalies.

Initial Dataset Preview

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
0	6	148	72	35	0	33.6	0.627	50	1	
1	1	85	66	29	0	26.6	0.351	31	0	
2	8	183	64	0	0	23.3	0.672	32	1	
3	1	89	66	23	94	28.1	0.167	21	0	
4	0	137	40	35	168	43.1	2.288	33	1	

Essential Data Inspection Commands

Dimensions and Column Names:

```
# Number of rows and columns
df.shape

# Returns: (768, 9)

# List of column names
df.columns

# Total elements
df.size
```

Sampling Data Records:

```
# First 10 rows
df.head(10)

# Last 5 rows
df.tail()

# 5 random samples
df.sample(5)
```

Best Practice

Checking `df.sample(5)` often provides a more unbiased view than `head()` or `tail()`, as raw datasets might be sorted by specific criteria.

Step 2: Checking Data Types with `df.info()`

`df.info()` provides a concise summary: record count, column count, non-null counts, data types (dtype), and memory usage.

```
df.info()
```

Common Data Types	Typical Interpretation
int64	Integer quantities (age, count, transaction IDs)
float64	Floating-point numbers (revenue, measurements, rates)
object	Text strings or mixed unstructured data
bool	Boolean flags (True/False)
datetime64	Timestamps and dates
category	Categorical variables with a fixed set of levels

Checking for Missing Values

Calculate total counts and percentages of missing entries per column:

```
# Count missing values per column
df.isna().sum() # Equivalent to df.isnull().sum()

# Calculate missing percentage per column
missing_rate = df.isna().mean() * 100
print(missing_rate)
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0
dtype: int64	

Crucial Insight: Implicit Missing Values

Missing data is not always encoded as NaN!

In production systems, missing values are frequently recorded using sentinel values:

0, -1, 999, "Unknown", "N/A", "?"

Example in Healthcare (diabetes.csv):

- Physiological measurements such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI contain values of **0**.
- Clinically, a living individual cannot have a blood pressure or glucose level of 0!
- These are **implicit missing values** that must be identified via domain knowledge.

Example in Business Analytics:

- Age = 0 or -1; Delivery_Time = 999 days; Revenue = 0.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection**
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Step 3: Descriptive Statistics with `df.describe()`

```
# Descriptive statistics for numerical variables  
df.describe()  
  
# Descriptive statistics for all variables (including categoricals)  
df.describe(include="all")
```

Key Diagnostic Indicators:

- **min and max:** Are values within plausible physical bounds? (e.g., Age having $\text{min} = -3$ or $\text{max} = 245$).
- **mean vs. 50% (median):**
 - If $\text{mean} \approx \text{median}$: Distribution is approximately symmetric.
 - If $\text{mean} \gg \text{median}$: Right-skewed distribution, driven by extreme positive values.

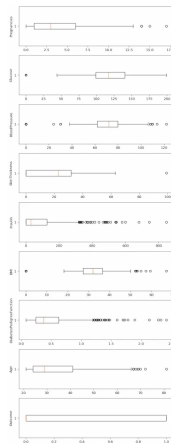
Step 4: Visualizing Outliers with Boxplots

A **Boxplot** is an indispensable tool for inspecting distribution shape and anomalies:

```
# Single feature boxplot
plt.figure(figsize=(8, 3))
plt.boxplot(df["Insulin"].dropna(), vert=False)
plt.xlabel("Insulin")
plt.title("Boxplot of Insulin")
plt.show()

# Multi-feature boxplots via subplots
numeric_cols = df.select_dtypes(include=np.number).columns
fig, axes = plt.subplots(len(numeric_cols), 1, figsize=(8, 2.5 * len(numeric_cols)))
for i, col in enumerate(numeric_cols):
    axes[i].boxplot(df[col].dropna(), vert=False)
    axes[i].set_title(col)
plt.tight_layout()
```

Boxplot Visualizations Across Numerical Features



Histograms & Distribution Assessment

Combining histograms with Kernel Density Estimation (KDE) reveals distribution skewness:

```
# Plotting with Pandas
df["Insulin"].hist(bins=30)
plt.xlabel("Insulin")
plt.ylabel("Frequency")
plt.show()

# Plotting with Seaborn (KDE included)
sns.histplot(data=df, x="Insulin", kde=True)
plt.title("Distribution of Insulin")
plt.show()
```

Insight

For heavily skewed features, log transformations or median-based scaling are generally more robust than mean-variance scaling.

Understanding Outliers

An **outlier** is an observation that lies an abnormal distance from other values in the sample.

Core Reality

Outliers are NOT inherently erroneous data!

Practical Examples:

- A VIP corporate client spending \$50,000 compared to an average of \$200.
- An international delivery shipment taking 45 days compared to a 3-day baseline.
- In **Fraud Detection**, rare fraudulent transactions are precisely the outliers we aim to detect!

Guiding Rule: The first priority is to **investigate the context and cause**, never delete outliers blindly.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method**
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Interquartile Range (IQR) Formula

The IQR method relies on quartiles, making it robust against extreme values:

- Q_1 (25th Percentile): 25% of the data falls below this value.
- Q_3 (75th Percentile): 75% of the data falls below this value.

Interquartile Range and Detection Thresholds

$$IQR = Q_3 - Q_1$$

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR$$

Observations falling outside [Lower Bound, Upper Bound] are flagged as candidate outliers.

Implementing the IQR Method in Python

```
# Compute Q1, Q3, and IQR
q1, q3 = np.percentile(df["Insulin"].dropna(), [25, 75])
iqr = q3 - q1

lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

print(f"Lower Bound: {lower:.2f} | Upper Bound: {upper:.2f}")

# Filter outlier records
outliers = df[(df["Insulin"] < lower) | (df["Insulin"] > upper)]
print("Total Outliers Detected:", len(outliers))

# If justifiable to filter:
clean_df = df[(df["Insulin"] >= lower) & (df["Insulin"] <= upper)].copy()
```

5 Questions to Ask Before Removing Outliers

Before discarding any outlier, ask:

- 1 Is the value **physically impossible** in the given business domain? (e.g., Age = -5 or 250).
- 2 Is it a known **data entry error** or logging glitch? (e.g., typing an extra zero).
- 3 Is there a **unit mismatch**? (e.g., grams vs. kilograms, cents vs. dollars).
- 4 Is the outlier the **primary object of interest**? (e.g., fraud, equipment failure).
- 5 Does removing it discard an **important customer segment**?

Alternative Treatment Strategies:

- Retain the data and adopt robust models (Decision Trees, Random Forests).
- Winsorization / Capping (clipping values to Lower/Upper bounds).
- Mathematical transformations (Log, Box-Cox) to compress variance.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values**
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Basic Missing Data Strategies

1. Deletion (Rows or Columns):

```
# Drop all rows containing missing values
df_clean = df.dropna()

# Drop rows missing specific features
df_clean = df.dropna(subset=["Age"])

# Drop columns missing > 60% of entries
df_clean = df.drop(columns=["Col"])
```

Only delete when missingness is negligible and random.

2. Statistical Imputation:

```
# Impute with Mean
df["Age"] = df["Age"].fillna(
    df["Age"].mean()
)

# Impute with Median (robust to outliers)
df["Income"] = df["Income"].fillna(
    df["Income"].median()
)

# Impute Categoricals with Mode
df["City"] = df["City"].fillna(
    df["City"].mode()[0]
)
```

SimpleImputer in Scikit-Learn

Scikit-Learn provides SimpleImputer for standardized, production-ready imputation:

```
from sklearn.impute import SimpleImputer

# Median imputation for numerical features
imputer_num = SimpleImputer(strategy="median")

# Mode imputation for categorical features
imputer_cat = SimpleImputer(strategy="most_frequent")

# Constant value imputation
imputer_const = SimpleImputer(strategy="constant", fill_value="Unknown")
```

Key Parameters:

- strategy: "mean", "median", "most_frequent", "constant".
- fill_value: Replacement value when strategy="constant".

Critical Warning: Data Leakage During Imputation

Data Leakage Prevention Principle

NEVER compute statistics (mean, median) across the entire dataset before splitting into train and test sets!

Training Data $\rightarrow$ Fit Parameter (e.g., Median) $\rightarrow$ Transform Both Train & Test

Why this is mandatory:

- The test set must simulate unseen future production data.
- Computing imputation values using the test set introduces information leakage, yielding overoptimistic test metrics that degrade in deployment.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)**
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Why Is Categorical Encoding Necessary?

Most machine learning algorithms (Linear/Logistic Regression, Neural Networks, SVM) operate strictly on numerical matrices. Hence, qualitative labels:

$$\text{Region} \in \{\text{"North"}, \text{"Central"}, \text{"South"}\}$$

must be converted into numerical binary representations.

Method 1: One-Hot Encoding via Pandas (pd.get_dummies)

```
# drop_first=True prevents multicollinearity (dummy variable trap)
df_encoded = pd.get_dummies(df, columns=["Region"], drop_first=True)
```

Region (Original)	Region_Central	Region_North	Region_South (Dropped if drop_first)
Central	1	0	0
North	0	1	0
South	0	0	1

OneHotEncoder in Scikit-Learn

For robust production pipelines, Scikit-Learn's OneHotEncoder is the gold standard:

```
from sklearn.preprocessing import OneHotEncoder

# handle_unknown='ignore' is crucial for unseen test labels
encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

# Learn categories from training set and transform
X_train_encoded = encoder.fit_transform(X_train[["Region"]])
X_test_encoded = encoder.transform(X_test[["Region"]])
```

Why is handle_unknown="ignore" Essential?

If an unseen category appears during testing (e.g., "Highlands"), `pd.get_dummies` would yield inconsistent column structures or crash, whereas `handle_unknown="ignore"` encodes all category columns with zeros safely.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution**
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Step 8: Correlation Analysis

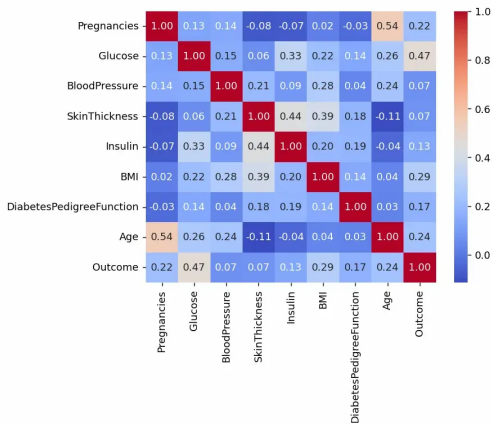
Pearson's linear correlation coefficient $r \in [-1, 1]$:

```
# Compute correlation matrix for numerical features
corr = df.corr(numeric_only=True)

# Visualize using Seaborn Heatmap
plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()

# Inspect correlation with target variable Outcome
corr["Outcome"].sort_values(ascending=False)
```

Feature Correlation with Target Variable



Key Takeaways:

- Glucose ($r = 0.47$) and BMI ($r = 0.29$) show the strongest positive correlation with diabetes diagnosis (Outcome).
- Age ($r = 0.24$) also correlates positively.
- **Caution:** High collinearity among predictors may cause **multicollinearity**.
- Low linear correlation does NOT imply a feature is useless, as the underlying relationship may be non-linear.

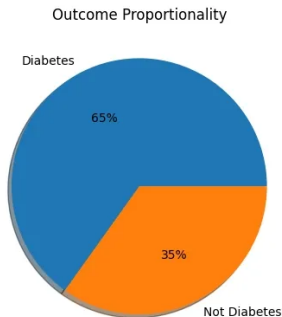
Step 9: Evaluating Target Distribution

For classification problems, auditing the class balance is mandatory:

```
# Count frequencies and proportions
print(df["Outcome"].value_counts())
print(df["Outcome"].value_counts(normalize=True))

# Visualize with a pie chart or countplot
counts = df["Outcome"].value_counts()
plt.pie(counts.values, labels=counts.index, autopct="%.1f%%")
plt.title("Outcome Distribution")
plt.show()
```

Target Class Balance in diabetes.csv



- Class 0 (Non-diabetic): 65.1% | Class 1 (Diabetic): 34.9%.
- Moderate imbalance is present. Under extreme imbalance (e.g., 99% vs 1%), raw accuracy becomes deceptive, requiring **Precision, Recall, F1-score, ROC-AUC**, and **Stratified Sampling**.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split**
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Step 10: Splitting Predictors (X) and Target (y)

```
# X contains input predictors (features)
X = df.drop(columns=["Outcome"])

# y contains the target label to predict
y = df["Outcome"]

print("Predictor Matrix Shape (X):", X.shape)
print("Target Vector Shape (y):", y.shape)
```

Significance:

- X contains information available at prediction time.
- y represents the ground-truth outcome.
- Ensure X never includes variables calculated after the target occurs (**Target Leakage**).

Step 11: Train/Test Data Partitioning

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% reserved for holdout evaluation
    random_state=42, # Reproducibility seed
    stratify=y # Preserves class proportions across splits
)
```

Key Argument Explanations:

- `test_size=0.2`: 80% used for model fitting, 20% reserved for objective testing.
- `random_state=42`: Guarantees identical data partitions across runs.
- `stratify=y`: Maintains identical target class ratios ($\approx 35\%$) across both train and test splits.

The Golden Rule: Why Split Before Scaling?

WRONG (Data Leakage)

```
scaler = StandardScaler()  
# Scaler fits on ENTIRE dataset  
X_scaled = scaler.fit_transform(X)  
  
# Split performed post-scaling  
X_tr, X_te = train_test_split(X_scaled)
```

→ Test set mean μ and variance σ^2 leak into the model training process!

CORRECT PRACTICE

```
# Partition train/test first  
X_tr, X_te, y_tr, y_te = (  
    train_test_split(X, y, test_size=0.2)  
)  
  
scaler = StandardScaler()  
# ONLY fit on Train  
X_tr_scaled = scaler.fit_transform(X_tr)  
# Test is ONLY transformed  
X_te_scaled = scaler.transform(X_te)
```

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)**
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Two Foundational Scaling Paradigms

1. Min-Max Scaling

(Normalization) Maps feature values into a bounded interval, typically $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Minimum value transforms to 0.
- Maximum value transforms to 1.
- Sensitive to extreme outliers because $x_{\min}$ and $x_{\max}$ anchor the scale.

2. Standardization (Z-score)

Centers data around mean 0 with standard deviation 1:

$$z = \frac{x - \mu}{\sigma}$$

- Does not constrain data to bounded limits.
- Less vulnerable to extreme outliers than Min-Max.
- Does not transform non-normal distributions into Gaussian shapes!

Min-Max Scaling Implementation & Output

```
from sklearn.preprocessing import MinMaxScaler
```

```
minmax_scaler = MinMaxScaler()
X_train_norm = minmax_scaler.fit_transform(X_train)
X_test_norm = minmax_scaler.transform(X_test)
```

```
array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.        ,
        0.50074516, 0.23441503, 0.48333333],
       [0.05882353, 0.42713568, 0.54098361, 0.29292929, 0.        ,
        0.39642325, 0.11656704, 0.16666667],
       [0.47058824, 0.91959799, 0.52459016, 0.        , 0.        ,
        0.34724292, 0.25362938, 0.18333333],
       [0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111,
        0.41877794, 0.03800171, 0.        ],
       [0.        , 0.68844221, 0.32786885, 0.35353535, 0.19858156,
        0.64232489, 0.94363792, 0.2        ]])
```

Standardization Implementation & Output

```
from sklearn.preprocessing import StandardScaler
```

```
standard_scaler = StandardScaler()  
X_train_std = standard_scaler.fit_transform(X_train)  
X_test_std = standard_scaler.transform(X_test)
```

```
array([[ 0.63994726,  0.84832379,  0.14964075,  0.90726993, -0.69289057,  
        0.20401277,  0.46849198,  1.4259954 ],  
       [-0.84488505, -1.12339636, -0.16054575,  0.53090156, -0.69289057,  
       -0.68442195, -0.36506078, -0.19067191],  
       [ 1.23388019,  1.94372388, -0.26394125, -1.28821221, -0.69289057,  
       -1.10325546,  0.60439732, -0.10558415],  
       [-0.84488505, -0.99820778, -0.16054575,  0.15453319,  0.12330164,  
       -0.49404308, -0.92076261, -1.04154944],  
       [-1.14185152,  0.5040552 , -1.50468724,  0.90726993,  0.76583594,  
        1.4097456 ,  5.4849091 , -0.0204964 ]])
```

Min-Max Scaling vs. Standardization

Evaluation Dimension	Min-Max Scaling	Standardization
Mathematical Formula	$\frac{X - X_{\min}}{X_{\max} - X_{\min}}$	$\frac{X - \mu}{\sigma}$
Output Range	Typically bounded $[0, 1]$	Unbounded $(-\infty, +\infty)$
Resulting Mean	Not necessarily 0	Equal to 0 on training set
Resulting Standard Dev.	Variable	Equal to 1 on training set
Sensitivity to Outliers	Very High	Moderate
Typical Applications	Bounded algorithms, image processing (0-255 pixels), k-NN	Ridge/Lasso regression, SVM, PCA, Neural Networks

Which Algorithms Require Feature Scaling?

REQUIRES Feature Scaling

Models relying on distance metrics or gradient descent optimization:

- k-Nearest Neighbors (k-NN)
- k-Means Clustering
- Support Vector Machines (SVM)
- Principal Component Analysis (PCA)
- Regularized Linear/Logistic Models
- Deep Neural Networks

DOES NOT Require Scaling

Tree-based partition algorithms:

- Decision Trees
- Random Forests
- Gradient Boosted Trees (XGBoost, LightGBM, CatBoost)

Rationale: Trees partition features monotonically and independently; splits are invariant to linear scale shifts.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn**
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment

Handling Mixed Datasets with ColumnTransformer

Real-world datasets contain both continuous numerical and categorical columns:

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Pipeline for numerical features: Median Imputation -> Standardization
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Pipeline for categorical features: Mode Imputation -> One-Hot Encoding
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Assemble combined preprocessor via ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, ["Age", "Income", "Order_Value"]),
    ("cat", categorical_transformer, ["Region", "Customer_Type"])
])
```

Full Model Integration via Pipeline

Encapsulate preprocessing and estimator into a unified object:

```
from sklearn.linear_model import LogisticRegression

# Bundle preprocessor and model estimator together
full_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

# Fit pipeline: Learned strictly from train set, zero leakage risk!
full_pipeline.fit(X_train, y_train)

# Inference on unseen test data
y_pred = full_pipeline.predict(X_test)
```

Major Pipeline Advantages:

- 1 Completely eliminates data leakage across validation folds.
- 2 Concise, readable, and maintainable codebase.
- 3 Seamless transition from development to production inference.

Session Outline

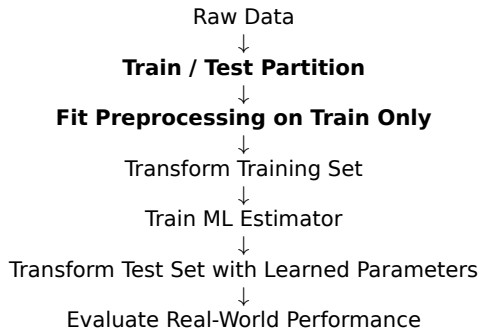
- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls**
- 12 Hands-On Exercises and Self-Assessment

Standard End-to-End Preprocessing Workflow

10-Step Robust Sequence:

- 1 **Understand:** Business domain and features.
- 2 **Inspect:** Examine shape, info, types.
- 3 **Explore (EDA):** Distributions, boxplots, correlations.
- 4 **Split:** Partition Train/Validation/Test sets.
- 5 **Impute:** Treat missing values from Train.
- 6 **Encode:** Transform categorical columns.
- 7 **Scale:** Standardize or normalize

Execution Flowchart



Common Preprocessing Traps (Part 1: Data & Outliers)

1 Automated Deletion of All Outliers:

- Flawed because outliers often represent valid high-value entities or crucial target signals (e.g., financial fraud).

2 Solely Relying on `isna()`:

- Misses implicit sentinel representations (0, $-1,999$, "?", "Unknown").

3 Scaling Target Labels (y):

- In classification, target labels are discrete categorical classes (0/1) and must never be scaled.

4 Discarding Features Due to Low Linear Correlation:

- Pearson correlation only reflects linear relationships; valuable non-linear patterns may be lost.

Common Preprocessing Traps (Part 2: Leakage & Evaluation)

5 **Fitting Scalers / Imputers Prior to Train/Test Split:**

- The most pervasive form of **Data Leakage**, distorting performance estimates.

6 **Selecting Features with Overly High Target Correlation:**

- Danger of **Target Leakage** (e.g., using Sales_Tax calculated directly from the forecast target Total_Sales).

7 **Inconsistent Processing of Train vs. Test Sets:**

- Induces distribution drift, leading to catastrophic inference failures in production.

Case Study: Customer Churn Prediction

Telecommunications Dataset Features:

- Customer_ID, Age, Monthly_Spending, Tenure_Months, Region, Support_Calls, Churn

Standard Preprocessing Steps:

- 1 Drop Customer_ID (arbitrary identifier with zero predictive power).
- 2 Detect Age = -1 → convert to NaN, then impute with median.
- 3 Clean Region: Harmonize "Hanoi", "Ha Noi", "HN" into "Hanoi".
- 4 Evaluate Support_Calls outliers via boxplots; retain them as high call volume strongly signals churn risk.
- 5 Separate X, y and partition with stratify= y .
- 6 Build Pipeline: Impute & scale numerical variables; One-hot encode categoricals.

Session Outline

- 1 Overview of Data Preprocessing
- 2 Steps 1 & 2 — Loading and Inspecting Data Structure
- 3 Steps 3 & 4 — Descriptive Statistics and Outlier Detection
- 4 Step 5 — Outlier Detection using the IQR Method
- 5 Step 6 — Handling Missing Values
- 6 Step 7 — Handling Categorical Variables (Encoding)
- 7 Steps 8 & 9 — Correlation Analysis and Target Distribution
- 8 Steps 10 & 11 — Feature Splitting and Train/Test Split
- 9 Step 12 — Feature Scaling (Min-Max vs. Standardization)
- 10 Step 13 — Building Preprocessing Pipelines with Scikit-Learn
- 11 General Workflow and Common Pitfalls
- 12 Hands-On Exercises and Self-Assessment**

Hands-On Coding Exercises

Exercise 1 (Data Inspection): Given DataFrame `df`, write code to compute the missing percentage of each column and display them sorted in descending order.

Exercise 2 (Missing Data): Write code to impute missing entries in `Income` using the median computed exclusively on `X_train`.

Exercise 3 (IQR Outlier Thresholds):

```
# Fill in the blanks to compute lower and upper bounds
q1 = df["Order_Value"].quantile(0.25)
q3 = df["Order_Value"].quantile(0.75)
iqr = _____
lower_bound = _____
upper_bound = _____
```

Concept Review & Quiz

Question 1. When a numerical feature is heavily right-skewed with severe positive outliers, which imputation statistic is most suitable?

- ☐ A. Mean
- ☐ B. Median
- ☐ C. Minimum
- ☐ D. Constant 0

Question 2. Which usage of StandardScaler complies strictly with machine learning best practices?

- ☐ A. `scaler.fit_transform(X_train)` and `scaler.fit_transform(X_test)`
- ☐ B. `scaler.fit(X)` followed by `train_test_split(X)`
- ☐ C. `scaler.fit_transform(X_train)` and `scaler.transform(X_test)`
- ☐ D. `scaler.transform(X_train)` and `scaler.fit(X_test)`

Lecture Summary

- **Data Preprocessing** forms the critical bedrock of all predictive modeling.
- **Missing Values:** Beware of implicit sentinels (0, -1, 999); standardize using SimpleImputer.
- **Outliers:** Diagnose using Boxplots and the IQR method; inspect context before deciding to filter or winsorize.
- **Categorical Encoding:** Utilize One-Hot Encoding with `handle_unknown="ignore"` to ensure inference safety.
- **Feature Scaling:** Differentiate Min-Max Scaling ($[0, 1]$ bounds) and Standardization ($\mu = 0, \sigma = 1$).
- **Leakage Prevention:** Always partition Train/Test first; only fit on Train and transform Test.
- **Pipelines & ColumnTransformer:** Scikit-Learn's core tools to build robust, modular, and leak-free workflows.

References

- ① Scikit-Learn Documentation. *Dataset transformations: preprocessing, imputation, pipelines*.
<https://scikit-learn.org/stable/modules/preprocessing.html>
- ② Shmueli, G., Bruce, P. C., Deokar, A. V., & Patel, N. R. (2023). *Machine Learning for Business Analytics: Concepts, Techniques and Applications in RapidMiner*. Wiley (Chapter 2).
- ③ Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- ④ McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.